CHANDIGARH UNIVERSITY
Discover. Learn. Empower.

UNIVERSITY INSTITUTE OF COMPUTING

PROJECT REPORT

ON

MATRIX OPERATIONS USING ARRAYS

Program Name: BCA – Data Science

Subject Name/Code: C Programming (Code: 26CAH-101)

Submitted by:
Name: Prachi
UID: 26BCD10321
Program: BCA – Data Science
Section: B

Submitted to:
Name: Ms. Kamaldeep Kaur

CHANDIGARH UNIVERSITY
Discover. Learn. Empower.

ABSTRACT

Introduction

This project is a menu-driven C program that performs three basic matrix operations: addition, multiplication, and transpose. A matrix is a rectangular arrangement of numbers in rows and columns. The program uses two-dimensional arrays to store matrix elements and functions to perform the required operations.

The user enters the dimensions and elements of the matrices, then selects an operation from the menu. The program checks whether the selected operation is possible for the given dimensions. It displays the resulting matrix in a clear row-and-column format.

Objectives

- To understand the concept of matrices and two-dimensional arrays in C.
- To perform matrix addition, multiplication, and transpose.
- To use functions to divide a program into smaller modules.
- To apply loops and conditional statements in matrix processing.
- To validate matrix dimensions before performing operations.

Techniques Used

- Two-dimensional arrays: Store matrix elements using rows and columns.
- Functions: Separate functions are used for input, display, addition, multiplication, and transpose.
- Nested for loops: Access and process each matrix element.
- if-else conditions: Check whether matrix operations are dimensionally valid.
- switch-case: Allows the user to select an operation from the menu.

System Configuration

- Operating System: Windows 10/11 or Linux
- Processor: Intel Core i3 or equivalent and above
- RAM: 4 GB minimum
- Development Environment: Code::Blocks, Visual Studio Code, Dev-C++, or any C IDE
- Compiler: GCC or another standard C compiler

CHANDIGARH UNIVERSITY
Discover. Learn. Empower.

PROJECT SUMMARY

Problem Statement

Build a C program to perform matrix operations such as addition, multiplication, and transpose using two-dimensional arrays.

Software Requirements

- C programming language
- C compiler such as GCC / MinGW
- Code editor or IDE such as Code::Blocks or Visual Studio Code

Program Working / Input

The program first displays a menu. The user chooses an operation and enters the required matrix dimensions and elements. The program then performs the selected operation and displays the result.

- 1 – Matrix Addition
- 2 – Matrix Multiplication
- 3 – Matrix Transpose
- 0 – Exit

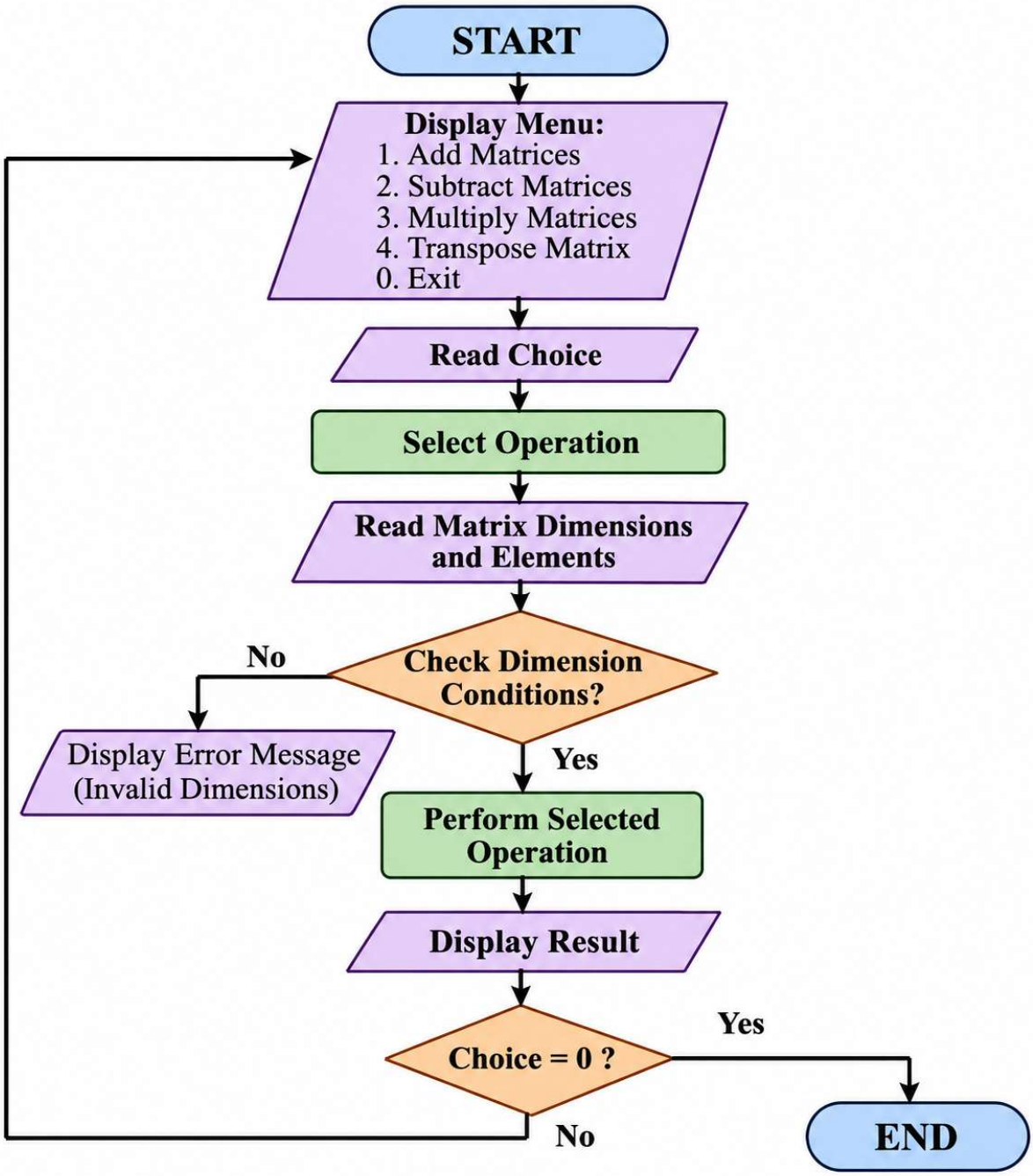
Rules for Matrix Operations

- Addition: Both matrices must have the same number of rows and columns.
- Multiplication: The number of columns in the first matrix must be equal to the number of rows in the second matrix.
- Transpose: Rows become columns and columns become rows.

Algorithm

1. Start the program and display the main menu.
2. Read the user's choice. If the choice is 0, terminate the program.
3. For addition, read the dimensions and elements of two matrices. Check that both dimensions are equal.
4. Add corresponding elements and store the result in a third matrix.
5. For multiplication, check that columns of the first matrix equal rows of the second matrix. Multiply rows by columns and add the products.
6. For transpose, read one matrix and interchange its row and column positions.
7. Display the resulting matrix.
8. Return to the menu and repeat until the user chooses Exit.

Flow of the Program



Source Code

```
#include <stdio.h>

#define MAX 10

void readMatrix(int a[MAX][MAX], int r, int c);
void displayMatrix(int a[MAX][MAX], int r, int c);
void addMatrices(int a[MAX][MAX], int b[MAX][MAX],
                 int r, int c);
void multiplyMatrices(int a[MAX][MAX], int b[MAX][MAX],
                      int r1, int c1, int c2);
void transposeMatrix(int a[MAX][MAX], int r, int c);

int main(void)
{
    int choice, r1, c1, r2, c2;
    int A[MAX][MAX], B[MAX][MAX];

    do
    {
        printf("\n===== MATRIX OPERATIONS =====\n");
        printf("1. Matrix Addition\n");
        printf("2. Matrix Multiplication\n");
        printf("3. Matrix Transpose\n");
        printf("0. Exit\n");
        printf("Enter your choice: ");
```

```
if (scanf("%d", &choice) != 1)
{
    printf("Invalid input.\n");
    return 1;
}

switch (choice)
{
    case 1:
        printf("Enter rows and columns of matrices: ");
        scanf("%d%d", &r1, &c1);

        if (r1 < 1 || r1 > MAX || c1 < 1 || c1 > MAX)
        {
            printf("Invalid dimensions. Use 1 to %d.\n", MAX);
            break;
        }

        printf("Enter elements of first matrix:\n");
        readMatrix(A, r1, c1);
        printf("Enter elements of second matrix:\n");
        readMatrix(B, r1, c1);
        addMatrices(A, B, r1, c1);
        break;
```

```
case 2:
    printf("Enter rows and columns of first matrix: ");
    scanf("%d%d", &r1, &c1);
    printf("Enter rows and columns of second matrix: ");
    scanf("%d%d", &r2, &c2);

    if (r1 < 1 || r1 > MAX || c1 < 1 || c1 > MAX ||
        r2 < 1 || r2 > MAX || c2 < 1 || c2 > MAX)
    {
        printf("Invalid dimensions. Use 1 to %d.\n", MAX);
        break;
    }

    if (c1 != r2)
    {
        printf("Multiplication not possible: ");
        printf("columns of A must equal rows of B.\n");
        break;
    }

    printf("Enter elements of first matrix:\n");
    readMatrix(A, r1, c1);
    printf("Enter elements of second matrix:\n");
    readMatrix(B, r2, c2);
    multiplyMatrices(A, B, r1, c1, c2);
    break;
```

```
        case 3:
            printf("Enter rows and columns of matrix: ");
            scanf("%d%d", &r1, &c1);

            if (r1 < 1 || r1 > MAX || c1 < 1 || c1 > MAX)
            {
                printf("Invalid dimensions. Use 1 to %d.\n", MAX);
                break;
            }

            printf("Enter matrix elements:\n");
            readMatrix(A, r1, c1);
            transposeMatrix(A, r1, c1);
            break;

        case 0:
            printf("Program closed. Thank you!\n");
            break;

        default:
            printf("Invalid choice. Select 0 to 3.\n");
    }

} while (choice != 0);
```

```
    return 0;
}

void readMatrix(int a[MAX][MAX], int r, int c)
{
    int i, j;
    for (i = 0; i < r; i++)
        for (j = 0; j < c; j++)
            scanf("%d", &a[i][j]);
}

void displayMatrix(int a[MAX][MAX], int r, int c)
{
    int i, j;
    for (i = 0; i < r; i++)
    {
        for (j = 0; j < c; j++)
            printf("%d\t", a[i][j]);
        printf("\n");
    }
}

void addMatrices(int a[MAX][MAX], int b[MAX][MAX],
                 int r, int c)
{
    int sum[MAX][MAX], i, j;
```

```
    for (i = 0; i < r; i++)
        for (j = 0; j < c; j++)
            sum[i][j] = a[i][j] + b[i][j];

    printf("Sum of matrices:\n");
    displayMatrix(sum, r, c);
}

void multiplyMatrices(int a[MAX][MAX], int b[MAX][MAX],
                      int r1, int c1, int c2)
{
    int product[MAX][MAX] = {0};
    int i, j, k;

    for (i = 0; i < r1; i++)
        for (j = 0; j < c2; j++)
            for (k = 0; k < c1; k++)
                product[i][j] += a[i][k] * b[k][j];

    printf("Product of matrices:\n");
    displayMatrix(product, r1, c2);
}

void transposeMatrix(int a[MAX][MAX], int r, int c)
{
    int trans[MAX][MAX], i, j;
```

```
    for (i = 0; i < r; i++)
        for (j = 0; j < c; j++)
            trans[j][i] = a[i][j];

    printf("Transpose of matrix:\n");
    displayMatrix(trans, c, r);
}
```

CHANDIGARH UNIVERSITY
Discover. Learn. Empower.

OUTPUT / SAMPLE RUNS

Sample Run 1 – Matrix Addition

Program output - Matrix Addition

```
===== MATRIX OPERATIONS =====
1. Matrix Addition
2. Matrix Multiplication
3. Matrix Transpose
0. Exit
Enter your choice: 1
Enter rows and columns of matrices: 2 3
Enter elements of first matrix:
1 2 3
4 5 6
Enter elements of second matrix:
10 20 30
40 50 60
Sum of matrices:
11      22      33
44      55      66

===== MATRIX OPERATIONS =====
1. Matrix Addition
2. Matrix Multiplication
3. Matrix Transpose
0. Exit
Enter your choice: 0
Program closed. Thank you!
```

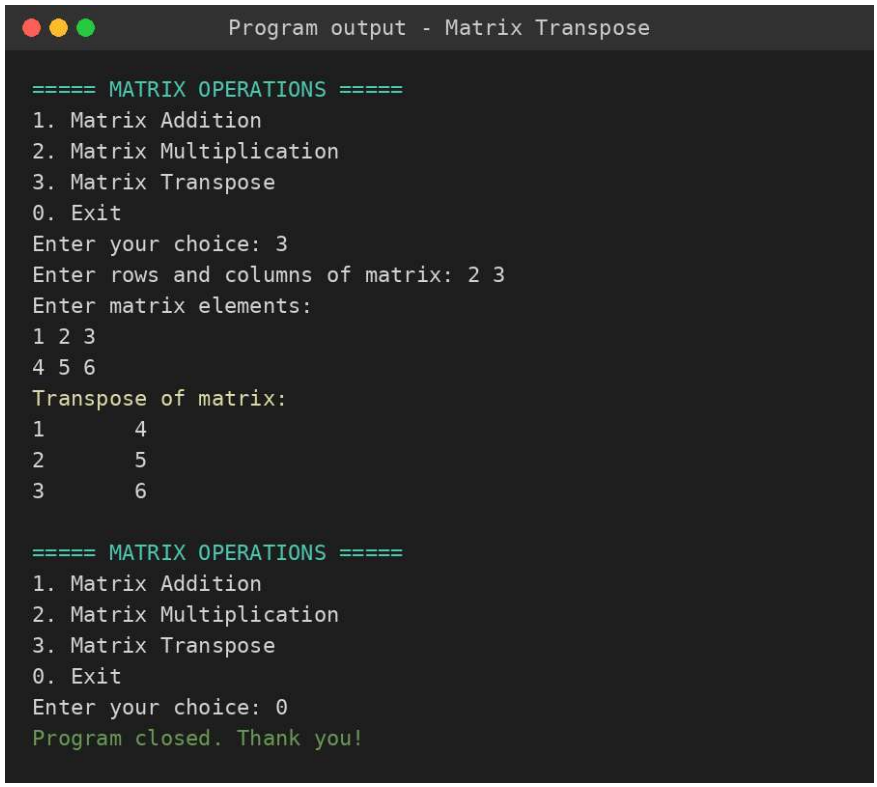
Sample Run 2 – Matrix Multiplication

Program output - Matrix Multiplication

```
===== MATRIX OPERATIONS =====
1. Matrix Addition
2. Matrix Multiplication
3. Matrix Transpose
0. Exit
Enter your choice: 2
Enter rows and columns of first matrix: 2 3
Enter rows and columns of second matrix: 3 2
Enter elements of first matrix:
1 2 3
4 5 6
Enter elements of second matrix:
7 8
9 10
11 12
Product of matrices:
58      64
139     154

===== MATRIX OPERATIONS =====
1. Matrix Addition
2. Matrix Multiplication
3. Matrix Transpose
0. Exit
Enter your choice: 0
Program closed. Thank you!
```

Sample Run 3 – Matrix Transpose



Sample Run 4 – Invalid Multiplication Dimensions

```
Enter your choice: 2
Enter rows and columns of first matrix: 2 3
Enter rows and columns of second matrix: 2 2
Multiplication not possible: columns of A must equal rows of B.
```

Sample Run 5 – Exit

```
Enter your choice: 0
Program closed. Thank you!
```

CONCLUSION

The Matrix Operations project successfully performs matrix addition, multiplication, and transpose using two-dimensional arrays in C. It demonstrates the use of nested loops, functions, switch-case, and conditional statements. Dimension checks help prevent invalid operations and make the program more reliable.

REFERENCES

- C programming class notes and study material.
- C standard library documentation for input and output functions.
- GCC documentation and compiler reference material.